

Senior Engineer Plan for Grid Mapping, Occupancy Grids, Obstacle Avoidance, and Path Planning

1. Problem Restatement and Constraints

The task is to implement autonomous navigation for a Webots ROSbot-like robot to drive from a blue pillar to a yellow pillar in minimum simulation time, starting from an unknown pose in an unknown environment.^[1]

The robot must not traverse green ground, must handle narrow and too-narrow passages, floating walls, and other obstacles, and cannot rely on the Webots supervisor or any extra sensors not already present.^[1]

The only available devices are four wheel motors, two front wheel position sensors, a 2D lidar, an RGB camera, and a depth camera, accessible via Webots C API `WbDeviceTags`.^[1]

As a senior autonomous robotics engineer, the approach is to design a **self-contained controller** that performs odometry, occupancy-grid mapping, pillar and green-ground perception, obstacle avoidance, global path planning, and a mission-level state machine within the constraints of Webots and the exam rules.

2. System Architecture Overview

2.1 Software Components

The controller is logically decomposed into the following modules, all running in a single Webots controller process:

- **Odometry module:** Computes the robot pose in a local world/odometry frame using wheel encoders.
- **Mapping module:** Maintains a 2D occupancy grid using lidar measurements and odometry; uses probabilistic log-odds updates similar to standard occupancy grid mapping.^{[2][3]}
- **Perception module:**
 - Detects blue and yellow pillars via color segmentation in RGB images.
 - Estimates pillar positions using depth data (or lidar) and transforms them into the map frame.
 - Detects green ground and marks it as forbidden either locally or in the grid.
- **Planning module:**
 - Global grid-based path planning using A* (or an improved variant) on the occupancy grid.^{[4][3]}

- Local reactive obstacle avoidance using lidar-based safety checks, inspired by classical methods such as Bug2 or Vector Field Histogram.[⁵][⁶][⁷]
- **Control module:** Converts path waypoints into left/right wheel velocities with closed-loop heading control.
- **Mission/state-machine module:** Coordinates phases: explore for blue, go to blue, explore for yellow, go to yellow.

These modules communicate through shared data structures (pose, grid, pillar list, current state) and run in a cyclic loop synchronized with Webots' timestep.

2.2 Coordinate Frames

Because no supervisor or global pose is available, the system defines a **local world/odometry frame** anchored at the starting pose.

All mapping and planning happen in this frame, and absolute coordinates are not required. The Webots lidar and camera are mounted at fixed, known offsets from the robot base, so their measurements are transformed using static extrinsics and the current odometry pose.[⁸]

3. Odometry from Motors and Position Sensors

3.1 Kinematic Model

The robot is treated as a **differential-drive platform**:

- Two main driven wheels (front left/right) with encoders (front wheel position sensors).
- Rear wheels driven with mirrored velocities to maintain stability, but not used for odometry.

Wheel radius R and wheel separation B are derived from the robot's geometrical properties within Webots (from CAD/URDF or measured in the world).[⁹]

3.2 Incremental Odometry Update

At each simulation step:

1. Read encoder values from `fl_pos_sensor` and `fr_pos_sensor`.
2. Compute increments:
 - $d\theta_L = \theta_L - \theta_{L,prev}$
 - $d\theta_R = \theta_R - \theta_{R,prev}$
3. Compute path and rotation increments:
 - $dS_L = R d\theta_L, dS_R = R d\theta_R$
 - $dS = (dS_L + dS_R)/2$
 - $d\psi = (dS_R - dS_L)/B$
4. Integrate pose in world frame with a standard second-order update:
 - $x \leftarrow x + dS \cos(\psi + d\psi/2)$

- $y \leftarrow y + dS \sin(\psi + d\psi/2)$
- $\psi \leftarrow \psi + d\psi$

This is the classic differential-drive odometry model used in occupancy grid mapping literature and Webots mapping demos, and is adequate for local navigation despite drift.^{[10][11][12]}

3.3 Velocity Commands

To drive the robot, the controller:

- Computes desired linear velocity v and angular velocity ω from path-tracking and obstacle avoidance.
- Converts to wheel angular velocities:
 - $\omega_L = (v - \omega B/2)/R$
 - $\omega_R = (v + \omega B/2)/R$
- Sets all four motors' velocities using `wb_motor_set_velocity`, with rear wheels mirroring front wheels.

Velocity limits are enforced to avoid unrealistic accelerations and to keep odometry and mapping stable.

4. Occupancy Grid Mapping with Lidar

4.1 Grid Specification

The environment is represented as a 2D occupancy grid where each cell stores the probability of being occupied.^{[3][2][10]}

Key design choices:

- Resolution: ~0.05–0.1 m per cell to capture narrow passages while keeping memory manageable.
- Size: configured to cover the known bounds of the exam arena, with margins.^[1]
- Representation: log-odds for each cell L_{ij} to support incremental Bayesian updates.^{[2][3]}

The mapping problem is to estimate $p(m_{ij} \mid z_{1:t}, x_{1:t})$ for each cell given all past lidar measurements and poses, with the usual assumption of conditional independence of cells.^[3]

4.2 Lidar Model and Ray Casting

Webots' Lidar device provides a fixed number of beams, range measurements, and FOV; for a 2D occupancy grid, only horizontal scanning is used.^[8]

For each time step:

1. Retrieve the scan: ranges r_k and beam angles ϕ_k in the robot frame.
2. For each beam:
 - If r_k is invalid or at max range, treat as "no hit": update only free cells along the ray and no occupied cell.

- Otherwise, compute endpoint in world frame via the current pose and then ray-cast from robot cell to endpoint cell.

Ray-casting (using Bresenham or similar) marks:

- Cells traversed before the endpoint as more likely free.
- The endpoint cell as more likely occupied.

This is consistent with classical occupancy grid mapping algorithms and has been used successfully in Webots-based mapping experiments.^{[12][10][2]}

4.3 Probabilistic Update (Log-Odds)

Each cell m_{ij} is updated using log-odds increments:

- For cells along the ray (free evidence):
 - $L_{ij} \leftarrow L_{ij} + l_{free} - L_0$
- For endpoint cell (occupied evidence):
 - $L_{ij} \leftarrow L_{ij} + l_{occ} - L_0$

where L_0 is the prior log-odds and l_{free}, l_{occ} are tuned to match lidar reliability.^{[2][3]}

Values are clamped to avoid numerical overflow and to prevent cells from becoming irreversibly occupied or free.

4.4 Handling Floating and Ground Walls

The world contains standing walls, floating walls, and walls lying on the ground.^[1]

To handle these with a 2D lidar:

- Mount the lidar at a height that captures obstacles important for collision (standing and floating walls).
- Accept that very low walls below the lidar plane may not be seen; rely on depth camera in the perception module to mark them as obstacles if necessary.
- Treat all lidar returns as obstacles in the grid without trying to classify wall types; from a navigation standpoint, any obstacle in the lidar plane is non-traversable.

Combining lidar and depth-based obstacle insertion (optional) maintains a more accurate 2D grid in complex 3D geometry.^{[6][13]}

5. Perception: Pillars and Green Ground

5.1 RGB and Depth Camera Interface

The controller enables both `camera_rgb` and `camera_depth` devices in Webots and periodically grabs frames.^{[14][8]}

Each iteration:

- Convert RGB buffer into a usable image structure (e.g. custom or OpenCV Mat).

- Convert depth buffer into a depth image giving per-pixel range.

5.2 Color Segmentation for Blue and Yellow Pillars

Color-based segmentation is performed in HSV space, following standard object detection by color tutorials used in Webots and OpenCV contexts.^{[15][16][17]}

For each RGB frame:

1. Convert image to HSV.
2. Define HSV ranges for blue and yellow:
 - `blue_low, blue_high`.
 - `yellow_low, yellow_high`.
3. Threshold to get binary masks:
 - `mask_blue = inRange(hsv, blue_low, blue_high)`.
 - `mask_yellow = inRange(hsv, yellow_low, yellow_high)`.
4. Apply morphological operations (erode/dilate) to reduce noise and merge blobs.
5. Find contours on each mask, select the largest contour above a minimum area threshold, and compute its centroid and bounding box.

This yields image-space pillar detections (u , v) with approximate size, which helps filter out spurious detections and track pillars over time.^{[17][15]}

5.3 Depth-Based 3D Localization of Pillars

To map detected pillars into the world/odometry frame:

1. Use depth image value at or near the centroid (u , v) to get range r .
2. Using camera intrinsics derived from Webots FOV and resolution, compute the bearing angle θ corresponding to pixel u .^[8]
3. Compute pillar coordinates in camera frame and then transform to robot base frame using the fixed camera-to-base transform.
4. Finally, transform to world/odometry frame with current odometry pose.

Repeated detections over time are fused (e.g. via moving average) to refine pillar positions and reduce noise.

When a pillar has been consistently observed, its world pose is locked in as a target for planning.

5.4 Green Ground Detection and Constraints

The exam forbids driving over green ground.^[1]

To enforce this:

- The RGB camera image is scanned in a bottom region (region of interest near the ground in front of the robot).

- Similar HSV segmentation is applied to detect green pixels, forming `mask_green`.
- For each green pixel in the ROI, its depth value is used to project into world coordinates, and corresponding grid cells are marked as `forbidden`.

In the occupancy grid, forbidden cells can be encoded as occupied or with a special label that the planner treats as non-traversable.

This ensures that global paths are planned to avoid green ground, and local obstacle checks also treat green areas as lethal.

6. Planning: Global Path and Local Obstacle Avoidance

6.1 Grid-Based Global Path Planning (A*)

Once pillar poses are available in the grid frame and the grid is sufficiently populated, a global planner uses A* to find shortest paths over free cells.^{[18][4][^3]}

Steps:

1. Convert continuous robot pose (x, y) and target pose (x_t, y_t) to grid indices.
2. Construct a graph implicitly from grid cells where each free cell has neighbors (4- or 8-connected). Neighbor connectivity may be extended or modified as in improved A* approaches to balance efficiency and path quality.^[^4]
3. Run A*:
 - Cost = path length (or weighted by proximity to obstacles for safety).
 - Heuristic = Euclidean or diagonal distance to goal.
4. Retrieve the sequence of grid cells from start to goal, then simplify (e.g. line-of-sight pruning) to generate a smaller set of waypoints.

This path is recomputed as needed when new map information arrives or when the robot's pose changes significantly.

6.2 Local Obstacle Avoidance Layer

Even with global planning, a local layer is required to cope with:

- Map incompleteness.
- Dynamic obstacles or mapping noise.
- Very narrow or too narrow passages.

The local layer uses lidar to monitor a safety zone in front of the robot and modifies commands from the path follower when necessary, following principles from Bug2, Vector Field Histogram, or similar reactive algorithms.^{[7][5][^6]}

Typical behavior:

- Compute minimum distance to obstacles in a forward cone.
- If distance < safety threshold, reduce speed or stop.

- If obstacles block the projected path, search for an open direction (e.g. heading with largest clear sector) and temporarily steer toward it while respecting the global goal direction.

This hybrid of global path planning and reactive obstacle avoidance is consistent with standard navigation architectures and has been applied with lidar in Webots and ROS-based systems.^[19]
[5][^6]

6.3 Controller: Tracking the Path

The control module tracks the global path by steering toward a selected "look-ahead" waypoint:

1. Select the closest waypoint ahead of the robot along the path.
2. Compute desired heading $\psi_d = \text{atan2}(y_w - y, x_w - x)$.
3. Compute heading error $e = \text{wrap}(\psi_d - \psi)$.
4. Set:
 - Linear velocity $v = v_{max}$ reduced near sharp turns or in confined spaces.
 - Angular velocity $\omega = k_p e$ with saturation.

This simple proportional controller is sufficient for low-speed navigation in a small arena. Wheel velocities are derived from v, ω as described in the odometry section.

7. Mission State Machine and Strategy

7.1 State Decomposition

The mission is structured as a finite state machine running in the main loop:

1. INITIALIZE
 - Reset pose and grid.
 - Start sensors.
2. EXPLORE_FOR_BLUE
 - No reliable blue pillar pose yet.
 - Perform exploration by periodically choosing exploration targets:
 - Simple strategy: radial or spiral patterns, or random goals within known free space.
 - More advanced: frontier-based exploration using the boundary between known free and unknown cells.^[^18]
 - Use global planner and local avoidance to reach exploratory waypoints.
 - Continuously run pillar and green detection.
 - Transition to GO_TO_BLUE when blue pillar pose is stable.
3. GO_TO_BLUE
 - Set goal near blue pillar (offset to avoid collision, oriented toward pillar).
 - Plan path on occupancy grid and execute.

- On arrival, refine blue pillar pose by focused observations.
- If yellow pillar already localized, transition to GO_TO_YELLOW; otherwise to EXPLORE_FOR_YELLOW.

4. EXPLORE_FOR_YELLOW

- Similar to EXPLORE_FOR_BLUE, but starting from vicinity of blue pillar.
- Continue until yellow pose is stable.

5. GO_TO_YELLOW

- From current pose (ideally near blue pillar), set goal near yellow pillar and plan/execute path.
- On arrival, stop and record mission time.

This state structure directly matches the graded exercise requirements: reach blue, then reach yellow with minimum time.^[1]

7.2 Exploration Policies under Resource Constraints

Because there is no supervisor or high-level mapping system, exploration uses only the occupancy grid built online and the robot's own sensors.

Frontier-based exploration is a natural fit for occupancy grids: identify cells on the boundary between known free and unknown, cluster them, and select representative frontier points as exploration goals.^{[18][3]}

This approach effectively covers the environment and tends to discover pillars and narrow corridors.

To reduce implementation complexity under exam constraints, a simplified exploration variant is acceptable (e.g. random waypoint selection biased toward unknown regions), as long as it integrates with the occupancy grid and avoids green ground and obstacles.

8. Handling Narrow and Too-Narrow Passages

The environment includes both traversable narrow passages and "too narrow" passages that the robot should avoid.^[1]

The occupancy grid and planner handle this through **configuration-space inflation**:

- Inflate obstacles by approximately half the robot's diameter in grid space (i.e. mark cells within that radius as occupied).
- Narrow corridors that are physically too small for the robot then appear as occupied or nearly blocked in the grid, and A* will not choose them.

This configuration-space expansion is standard practice in grid-based path planning and is supported by many navigation stacks.^{[10][3]}

9. Avoiding Green Ground in Planning and Control

Avoiding green ground is both a perception and planning problem.

The plan is to treat green cells as non-traversable, just like obstacles:

- As the perception module projects green regions into world coordinates, the corresponding occupancy grid cells are assigned a "forbidden" status.
- For planning, forbidden cells are treated as occupied—A* simply does not traverse them.
- For local control, if green is detected directly in front of the robot at close depth, the reactive layer triggers a turn away even if the global path would have passed through that region.

This dual approach increases robustness in case of mapping latency or perception noise.

10. Testing, Tuning, and Performance Considerations

10.1 Incremental Testing Plan

A realistic engineering plan is to develop and verify components in stages:

1. **Odometry only:** Move robot with manually defined velocities, log and visualize odometry to ensure a reasonable trajectory.
2. **Lidar visualization:** Plot raw scan in robot frame and transformed into world frame to validate angles and pose transforms.^{[14][8]}
3. **Static occupancy grid:** Build a map while teleoperating slowly; check if the grid matches the environment in Webots, similar to mapping tutorials used in Webots and GMapping demonstrations.^{[11][10][^2]}
4. **Pillar detection:** Place individual pillars and verify that color segmentation and depth projection produce accurate pillar positions.
5. **Green detection:** Verify that green ground in front is correctly found and projected into the grid.
6. **Local obstacle avoidance:** Without global planner, command the robot toward a wall and check that the reactive layer prevents collisions.^{[6][7]}
7. **A planner*:** On a known, static occupancy grid, test A* paths between manually chosen start and goal cells.
8. **Full mission:** Run the state machine, observe exploration, detection of blue, navigation to blue, exploration for yellow, and final navigation.

10.2 Tuning Parameters

Critical parameters include:

- Grid resolution and size (trade-off between accuracy and memory/CPU).^{[3][2]}
- Lidar update rates and thresholds (`l_free`, `l_occ`).^{[10][2]}
- Exploration goal selection frequency vs. map stability.

- A* heuristic weight and neighbor structure (potential improvements similar to advanced A* variants can reduce search effort).^[4]
- Safety distances for obstacle avoidance and green ground margins.
- Controller gains k_p and velocity limits.

10.3 Performance and Time Optimization

Since grading considers time and collisions as performance metrics, the plan aims for:

- Efficient exploration that minimizes detours (frontiers near pillars prioritized when pillars are partially detected).
- Direct but safe paths between pillars, avoiding unnecessary oscillations.
- Avoidance of collisions and green-ground incursions by conservative safety thresholds.

11. Compliance with Exam Rules

The exam specifies that all solutions must be implemented by the students themselves and that any external code must be explicitly referenced, with non-LLM code predating 2025 and properly cited in comments.^[1]

Therefore, while the architectural plan leverages standard methods from occupancy grids, A*, and reactive obstacle avoidance, actual code for ray-casting, log-odds updates, A*, and color segmentation needs to be written by the student or clearly attributed when any external snippets are adapted.^{[2][4][3]}

12. Summary of the Plan

The overall plan as a senior autonomous robot engineer is to:

- Use differential-drive odometry from front wheel encoders to define a local world frame.
- Build a 2D probabilistic occupancy grid in this frame using Webots lidar and log-odds updates.
- Detect blue and yellow pillars and green ground using HSV color segmentation and depth projection, mapping them into the grid.
- Plan global paths on the occupancy grid with A*, using configuration-space inflation to respect robot size and too-narrow passages, and treating green ground as forbidden.
- Add a lidar-based reactive obstacle avoidance layer to handle local hazards and map imperfections.
- Coordinate behaviors through a mission state machine that first explores and finds the blue pillar, then the yellow pillar, and finally navigates from blue to yellow with minimum time.

This architecture is fully compatible with the available devices and exam constraints and is grounded in established methods for occupancy grid mapping, grid-based planning, and lidar-based obstacle avoidance in Webots and general mobile robotics.^{[6][10][4][3][2][1]}

References

1. [Modularbeit.pdf](#) - Thomas Nierhoff
Autonomous Robots
1
Welcome to the graded exercise
Autonomous Robots
Prof. Dr.-Ing. ...
2. [Navigation of simulated mobile robots in the Webots ...](#)
3. [Lab03 - Grid and Graph based Path Planning - CourseWare Wiki](#)
4. [Research on path planning of mobile robots based on improved A ...](#) - To address the issues of low search efficiency, excessive node expansion, and the presence of redund...
5. [Obstacle avoiding robot | MoveBase using Lidar | Webots ROS | Robotic Software PicoDegree | Part 6](#) - Wish to get into the shoes of a Robotics Software Engineer and see the complete cycle of mobile robo...
6. [Obstacle avoidance for mobile robots using lidar sensors](#)
7. [Obstacle Avoidance Robot Using IR Sensor in Webots | Webots Tutorial 2 | Webots 2023 in Python](#) - Lets learn to create an obstacle avoiding robot in Webots.

☒TIMINGS

0:00 Intro

0:48 Create Webots...

8. [Lidar - Webots documentation](#) - The Lidar node is used to model a robot's on-board lidar (laser-scanner). The lidar measures depth i...
9. [Webots: ROSbot XL + SLAM Toolbox + Navigation2 - Husarion](#) - This guide demonstrates how to run the ROSbot XL simulation in Webots, create a map of the unknown w...
10. [GMapping | ROS with Webots | Robotic Software PicoDegree | Part 4 | Best mapping package](#) -
0:15 Introduction
02:35 Glimpse of GMapping
3:59 Implementation
08:57 Start GMapping
10:52 Mistake
...
11. [Map a New Environment by Driving Around with a Joystick or...](#)
12. [Webots hands-on II.5: Mapping overview](#) - This is a code walk-through from the specialization "Introduction to Robotics with Webots" on Course...
13. [3D mapping using OctoMap | Webots | Pointcloud](#) - #octomap #octovis #pointcloud #computervision

OctoMap tutorial: <https://octomap.github.io/>

P...

14. [Webots documentation: Technical-Questions 2019](#) - I want to make occupancy grid map by lidar in webots. my webots version is 7.1.0, so there is not li...
15. [Color Detection & Object Tracking - OpenCV Tutorial C++](#) - OpenCV Tutorials for beginners of image processing and computer vision. Learn basic concepts with lo...
16. [How to Recognize objects using camera on Webots](#) - I am trying to use the camera on <https://robotbenchmark.net/> for the Highway Driving, but I have som...
17. [Visual Object Recognition | Camera in Webots Simulator | \[Tutorial 15\]](#) - Interested in Robotics, Simulation, and can feel a spark..!!??
Wanna have fun with camera mounted on...
18. [Eindhoven University of Technology](#)
19. [Master Thesis](#)